

Keyboard Control

1. Course Content

1. Control the robot's movement using the keyboard and understand the principles. After running the program, publish velocity topics via keyboard keys to control the robot chassis movement.

2. Preparations

2.1 Content Description

This lesson uses the Jetson Orin NX as an example. For Raspberry Pi boards, you need to open a terminal on the host machine, enter the command to get into the Docker container, and then input the commands mentioned in this lesson in the terminal within the Docker container. A tutorial on how to enter the Docker container from the host can be found in this product's guide [00.Configuration and Operation Guide]-[3.Enter the Docker (For RPI5)]. For ORIN and RDK boards, you can directly open a terminal and enter the commands mentioned in this lesson.

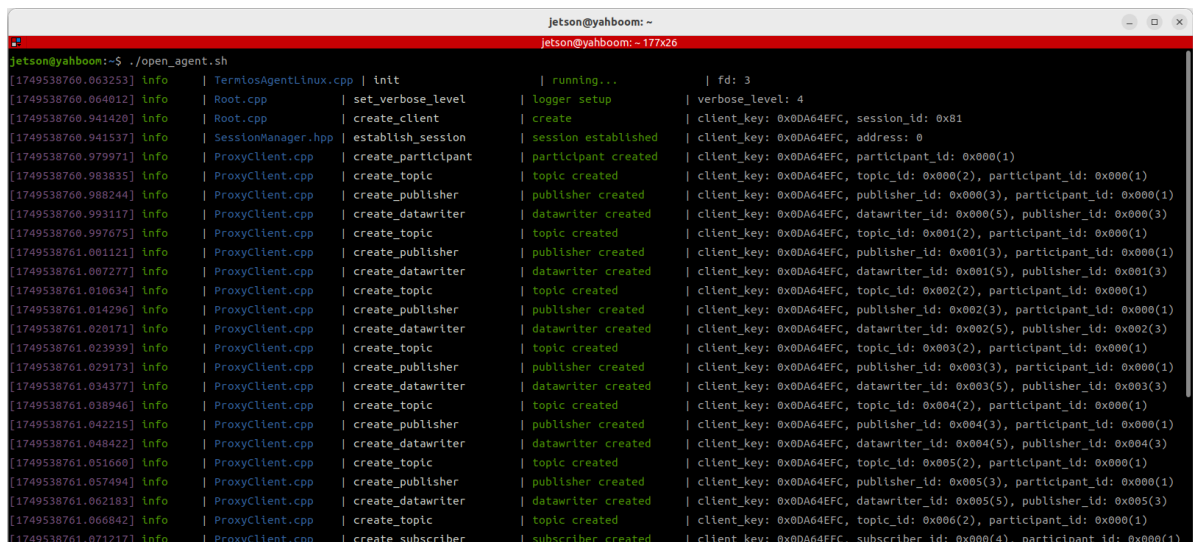
2.2 Start the Agent

Note: If it is already running, you do not need to start it again.

Enter the command in the vehicle's terminal:

```
start_agent
```

The terminal will print the following information, indicating a successful connection:



```
Jetson@yahboom: ~  
Jetson@yahboom: ~177x26  
Jetson@yahboom:~$ ./open_agent.sh  
[1749538760.063253] Info | TermiosAgentLinux.cpp | init | running... | fd: 3  
[1749538760.064012] Info | Root.cpp | set_verbose_level | logger setup | verbose_level: 4  
[1749538760.941420] Info | Root.cpp | create_client | create | client_key: 0x0DA64EFC, session_id: 0x81  
[1749538760.941537] Info | SessionManager.hpp | establish_session | session established | client_key: 0x0DA64EFC, address: 0  
[1749538760.979971] Info | ProxyClient.cpp | create_participant | participant created | client_key: 0x0DA64EFC, participant_id: 0x000(1)  
[1749538760.983835] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x000(2), participant_id: 0x000(1)  
[1749538760.988244] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x000(3), participant_id: 0x000(1)  
[1749538760.993117] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x000(5), publisher_id: 0x000(3)  
[1749538760.997675] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x001(2), participant_id: 0x000(1)  
[1749538761.001121] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x001(3), participant_id: 0x000(1)  
[1749538761.007277] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x001(5), publisher_id: 0x001(3)  
[1749538761.010634] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x002(2), participant_id: 0x000(1)  
[1749538761.014296] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x002(3), participant_id: 0x000(1)  
[1749538761.020171] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x002(5), publisher_id: 0x002(3)  
[1749538761.023939] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x003(2), participant_id: 0x000(1)  
[1749538761.029173] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x003(3), participant_id: 0x000(1)  
[1749538761.034377] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x003(5), publisher_id: 0x003(3)  
[1749538761.038946] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x004(2), participant_id: 0x000(1)  
[1749538761.042215] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x004(3), participant_id: 0x000(1)  
[1749538761.048422] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x004(5), publisher_id: 0x004(3)  
[1749538761.051660] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x005(2), participant_id: 0x000(1)  
[1749538761.057494] Info | ProxyClient.cpp | create_publisher | publisher created | client_key: 0x0DA64EFC, publisher_id: 0x005(3), participant_id: 0x000(1)  
[1749538761.062183] Info | ProxyClient.cpp | create_datawriter | datawriter created | client_key: 0x0DA64EFC, datawriter_id: 0x005(5), publisher_id: 0x005(3)  
[1749538761.066842] Info | ProxyClient.cpp | create_topic | topic created | client_key: 0x0DA64EFC, topic_id: 0x006(2), participant_id: 0x000(1)  
[1749538761.071217] Info | ProxyClient.cpp | create_subscriber | subscriber created | client_key: 0x0DA64EFC, subscriber_id: 0x000(4), participant_id: 0x000(1)
```

3. Run the Example

3.1 Start Keyboard Control

For Raspberry Pi 5 users, if the ROS container is not started, you need to start it first,

```
bringup_m3
```

Open a terminal inside the container,

```
exec_m3
```

Shortcut command:

```
keyboard_control
```

Full launch command:

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

```
jetson@yahboom:~$ ros2 run teleop_twist_keyboard teleop_twist_keyboard

This node takes keypresses from the keyboard and publishes them
as Twist/TwistStamped messages. It works best with a US keyboard layout.
-----
Moving around:
  u    i    o
  j    k    l
  m    ,    .

For Holonomic mode (strafing), hold down the shift key:
-----
  U    I    O
  J    K    L
  M    <    >

t : up (+z)
b : down (-z)

anything else : stop

q/z : increase/decrease max speeds by 10%
w/x : increase/decrease only linear speed by 10%
e/c : increase/decrease only angular speed by 10%

CTRL-C to quit

currently:      speed 0.50      turn 1.00
█
```

3.2 Key Control Description

3.2.1 Direction Control

Key(s)	Function
i or I	Forward
,	Backward
j or J	Turn Left/Strafe Left (lowercase for turn, uppercase for strafe)
l or L	Right Turn/Strafe Right (lowercase for turn, uppercase for strafe)
k	Stop Moving

3.2.2 Speed Control

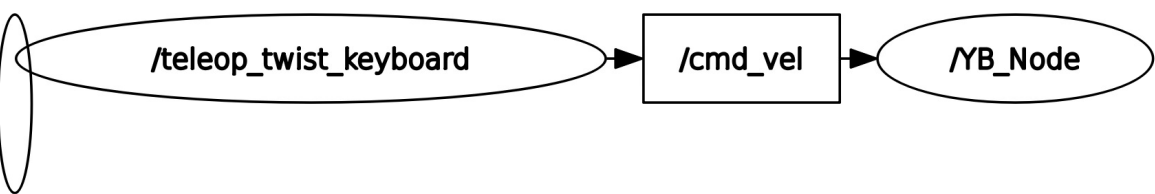
Key	Speed Change	Key	Speed Change
q	Increase linear and angular speed by 10%	z	Decrease linear and angular speed by 10%
w	Increase only linear speed by 10%	x	Decrease only linear speed by 10%
e	Increase only angular speed by 10%	c	Decrease only angular speed by 10%
t	Switch linear speed between X and Y axis	s	Stop keyboard control

4. Source Code Analysis

4.1 View Node Relationship Graph

Open a terminal and enter the command:

```
ros2 run rqt_graph rqt_graph
```



From the node relationship graph, we can see:

teleop_twist_keyboard: Controls the robot chassis by publishing to the `/cmd_vel` topic.

/YB_Node: The robot chassis node, which subscribes to the `/cmd_vel` topic and uses inverse kinematics to calculate the speed of each wheel to control the car's movement.

4.2 View Topic Messages and Message Types

Open a terminal and enter the command:

```
ros2 topic echo /cmd_vel
```

When controlling the robot chassis with the keyboard, you can see from the topic's printed messages that data is being published to the `/cmd_vel` topic.

```
x: 0.2
y: 0.0
z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
---
linear:
  x: 0.2
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
---
```

Enter the following command to view the message type of the `/cmd_vel` topic:

```
ros2 topic info /cmd_vel
```

From the `Type`, you can see that the message type for the `/cmd_vel` topic is `geometry_msgs/msg/Twist`. Enter the following command to view the composition of the `geometry_msgs/msg/Twist` message type:

```
ros2 interface show geometry_msgs/msg/Twist
```

```
jetson@yahboom:~$ ros2 interface show geometry_msgs/msg/Twist
# This expresses velocity in free space broken into its linear and angular parts.

Vector3 linear
  float64 x
  float64 y
  float64 z
Vector3 angular
  float64 x
  float64 y
  float64 z
jetson@yahboom:~$
```

From the composition of the message type above, it can be seen that the robot chassis is controlled by two vector groups: `linear` (linear velocity) and `angular` (angular velocity). Each data point is a `float64` type. Below is an explanation of each data field:

- **linear**
 - `float64 x`: linear velocity on the x-axis
 - `float64 y`: linear velocity on the y-axis
 - `float64 z`: linear velocity on the z-axis

- **angular**
 - `float64 x`: angular velocity on the x-axis
 - `float64 y`: angular velocity on the y-axis
 - `float64 z`: angular velocity on the z-axis

Since the robot chassis can only move in a 2D plane, only `linear.x` (x-axis linear velocity), `linear.y` (y-axis linear velocity), and `angular.z` (z-axis angular velocity) are actually published when controlling the robot with the keyboard.

4.3 Source Code Analysis

Source code path:

```
~/yahboomM3_ws/code_ws/src/m3_common_demo/m3_common_demo/keyboard_controller/teleop_twist_keyboard.py
```

4.3.1 Published Topic

```
pub = rospy.Publisher('cmd_vel', Twist, queue_size=1)
```

You just need to package the speed and publish it via `pub.publish(twist)`. The chassis's speed subscriber will then receive the speed data and drive the car.

4.3.2 Movement and Speed Dictionaries

- The movement dictionary mainly stores characters related to direction control:

```
moveBindings = {
    'i': (1,0,0,0),
    'o': (1,0,0,-1),
    'j': (0,0,0,1),
    'l': (0,0,0,-1),
    'u': (1,0,0,1),
    ',': (-1,0,0,0),
    '.': (-1,0,0,1),
    'm': (-1,0,0,-1),
    'O': (1,-1,0,0),
    'I': (1,0,0,0),
    'J': (0,1,0,0),
    'L': (0,-1,0,0),
    'U': (1,1,0,0),
    '<': (-1,0,0,0),
    '>': (-1,-1,0,0),
    'M': (-1,1,0,0),
    't': (0,0,1,0),
    'b': (0,0,-1,0),
}
```

- The speed dictionary mainly stores characters related to speed control:

```

speedBindings={
    'q':(1.1,1.1),
    'z':(.9,.9),
    'w':(1.1,1),
    'x':(.9,1),
    'e':(1,1.1),
    'c':(1,.9),
}

```

4.33 Get Current Key Information

```

def getKey():
    tty.setraw(sys.stdin.fileno())
    select.select([sys.stdin], [], [], 0)
    key = sys.stdin.read(1)
    termios.tcsetattr(sys.stdin, termios.TCSADRAIN, settings)
    return key

```

4.34 Check Key Value and Publish /cmd_vel Speed Topic

```

def main(args=None):
    if args is None:
        args = sys.argv

    rclpy.init(args)
    node = rclpy.create_node('teleop_twist_keyboard')

    pub = node.create_publisher(Twist, 'cmd_vel', qos_profile_default)

    speed = 0.5
    turn = 1.0
    x = 0
    y = 0
    z = 0
    th = 0
    status = 0

    try:
        print(msg)
        print(vels(speed,turn))
        while(1):
            key = getKey()
            if key in moveBindings.keys():
                x = moveBindings[key][0]
                y = moveBindings[key][1]
                z = moveBindings[key][2]
                th = moveBindings[key][3]
            elif key in speedBindings.keys():
                speed = speed * speedBindings[key][0]
                turn = turn * speedBindings[key][1]

            print(vels(speed,turn))
            if (status == 14):
                print(msg)
                status = (status + 1) % 15
            else:

```

```

        x = 0
        y = 0
        z = 0
        th = 0
        if (key == '\x03'):
            break

        twist = Twist()
        twist.linear.x = x*speed; twist.linear.y = y*speed;
twist.linear.z = z*speed;
        twist.angular.x = 0.0; twist.angular.y = 0.0; twist.angular.z =
th*turn
        pub.publish(twist)

    except:
        # print(e)
        pass

    finally:
        twist = Twist()
        twist.linear.x = 0.0; twist.linear.y = 0.0; twist.linear.z = 0.0
        twist.angular.x = 0.0; twist.angular.y = 0.0; twist.angular.z = 0.0
        pub.publish(twist)

    termios.tcsetattr(sys.stdin, termios.TCSADRAIN, settings)

```